

1. Importancia de ordenar/buscar en este tipo de logs.

Con esta evidencia pude entender mejor la importancia de ordenar datos, sobre todo cuando son muchos registros. La verdad no es la primera vez que trabajaba con algo así ya que en verano estuve desarrollando un software con un registro similar donde ocupaba filtrar y ordenar por fechas y horas pero claro en ese momento todo lo hice con inteligencia artificial. Ahora el hacerlo desde cero y ver cómo funciona el fondo de eso fue interesante y retador la verdad, el tener que pensar en el funcionamiento y también las cosas que pueden salir mal fue complejo.

Al principio pensaba que ordenar los logs nadamas era como un requisito del programa para que se viera mejor, pero al trabajar más con ello entendí que hace mucho más fácil ver y analizar la información y saber que paso en un periodo específico, como esto nos ayuda a tener mucho más control.

De esta manera, en lugar de tener que revisar todos los logs, uno por uno hasta ubicar alguna fecha u hora específica puedo ingresar una fecha inicial y una final para así obtener registros dentro de un periodo. Para hacer esto use la búsqueda binaria la cual aprovecha que los datos ya están ordenados para así encontrar los límites del rango de una manera más eficiente.

Otra cosa que apliqué aquí que ya había aplicado en mi trabajo de verano fue considerar casos específicos, solo que ahora yo tuve que crear los ifs lo cual fue algo tedioso. Estos casos fueron para fechas que no existen o registros que tienen la misma fecha y hora, es por esto que el programa valida las fechas ingresadas y la búsqueda incluye todos los logs que se encuentran en el rango, aunque haya timestamps repetidos.

2. Comparación conceptual de los algoritmos elegidos (fortalezas, debilidades y casos de uso), incluyendo por qué uno de ellos se comporta distinto en datos casi ordenados.

En la evidencia Estuve trabajando con diferentes algoritmos ordenamiento y algo que pude ver y entender mejor es que pues todos pueden llegar a un mismo resultado pero no todos trabajan de la misma manera ni tienen el mismo rendimiento antes de hacer las pruebas conocer las diferencias de manera teórica pero al aplicarlos a la misma cantidad de logs pude ver mucho más claro cómo estaba cambiando su comportamiento.

Bubble Sort y Selection Sort son algoritmos que son fáciles para mí de entender, porque básicamente solamente comparan elementos y los van acomodando poco a poco. Una ventaja de estos es que su lógica es más fácil de seguir, pero una desventaja que tienen es que su complejidad es de $O(n^2)$, por lo cual cuando tenemos más datos, pues también el trabajo que tienen que hacer aumenta. Entonces, como ahorita estábamos trabajando con más de 6 mil logs, sí se notó ese trabajo.

Insertion Sort también puede tener una complejidad de $O(n^2)$, pero para este depende mucho de cómo estén acomodados los datos inicialmente. Cuando los datos están muy desordenados puede tener que hacer muchos movimientos, pero cuando ya están casi ordenados no tiene que hacer tanto trabajo y puede llegar a tener un mejor caso de $O(n)$. Eso fue algo que pude estar comprobando al utilizar los dos archivos, el 1 y el 2, y me quedó mucho más claro al verlo.

También tenemos Merge Sort, que tiene una complejidad de $O(n \log n)$ el cual mantiene un rendimiento más constante sin importar el orden inicial de los datos. Pero por otro lado, Quick Sort puede llegar a ser muy rápido pero depende en cómo se escoge el pivote, como en mi caso use el último elemento como pivote esto hizo que su rendimiento fuera diferente en cada archivo.

Finalmente usé Shell Sort como un algoritmo extra que había agregado en la actividad 1.5. Este algoritmo utiliza saltos para comparar y acomodar elementos que se encuentran más separados y después va reduciendo esa distancia. Entonces, algo que aprendí al comparar todos estos algoritmos es que no solamente importa cuál algoritmo se utiliza, sino también cómo vienen originalmente los datos que estamos intentando ordenar.

3. Interpretación de sus mediciones, contrastadas con su predicción inicial y con la complejidad teórica.

Al estar comparando mis predicciones iniciales con los resultados que iba obteniendo, en general sí coincidieron con lo que esperaba según la complejidad de cada algoritmo. En el archivo desordenado, algoritmos como Swap, Bubble y Selection fueron más lentos, mientras que Merge y Quick tuvieron tiempos mucho menores.

Lo que más me llamó la atención fue Insertion Sort, ya que pasó de 3.42 segundos en el archivo desordenado a 0.025 segundos en el archivo casi ordenado. Esto me ayudó como a entender y comprobar que este algoritmo puede ser mucho más eficiente cuando los datos ya traen cierto orden. Esto ya lo había explicado en mi conclusión de las pruebas.

También pasó algo interesante con Quick Sort, pero al contrario. En el archivo desordenado tardó aproximadamente 0.082 segundos, mientras que en el casi ordenado subió a 1.83 segundos. Esto tiene que ver con que utilicé el último elemento como pivote, lo que afectó su rendimiento cuando los datos ya estaban casi ordenados.

Con estas mediciones pude ver de una manera más clara que la complejidad teórica sí ayuda a predecir el comportamiento de los algoritmos, pero también importa mucho cómo están acomodados originalmente los datos.

4. Uso de IA: 2-3 prompts clave que utilizaron, qué modificaron del código sugerido, y — obligatorio — al menos un caso concreto en el que la IA se equivocó o sugirió algo algorítmicamente incorrecto o ineficiente (por ejemplo, búsqueda lineal en vez de binaria, un ordenamiento inestable donde importaba estabilidad, un error de límites en el rango de fechas) y cómo lo detectaron y corrigieron.

En la evidencia usé principalmente ChatGPT y Copilot como herramientas de apoyo. ChatGPT lo estuve usando para que me explicara algunos conceptos con los cuales todavía tenía un poco de dudas. Quería entender mejor ciertos algoritmos, como Quick Sort, y también el concepto de la búsqueda binaria, también lo utilicé para discutir ideas de cómo podía empezar ciertas funciones o partes del código y cuál podía ser una buena manera de implementarlas. Además, algunas veces le pedí que me ayudara a detectar errores.

Por otro lado, Copilot me estuvo ayudando más con la sintaxis mientras escribía el código, por ejemplo con la indentación, mantener una estructura y algunos detalles de sintaxis.

Algunos de los prompts que utilicé fueron:

Prompt 1:

“No entiendo muy bien cómo funciona la búsqueda binaria. ¿Me puedes explicar paso a paso cómo se hace y por qué funciona para encontrar un rango? Explicámelo sencillo y dame ejemplos.”

Prompt 2:

“¿Me puedes ayudar a revisar mi Insertion Sort? Porque no estoy entendiendo muy bien el while y creo que tengo un error con los índices. No me muevas mi código ni me des el código corregido, solamente ayúdame a entender qué está mal y dame como pistas para corregirlo.”

Como se puede ver en los prompts, no buscaba copiar todo lo que me daba la inteligencia artificial. Especifique que no quería que me diera el código, sino ir entendiendo poco a poco, como si tuviera un maestro a mi lado diciéndome si iba bien o estaba haciendo algo mal. Así es como me he estado manejando en esta clase, ya que la verdad se me ha dificultado un poco. También estudié los conceptos y las complejidades de los algoritmos para poder entender mejor lo que estaba haciendo.

Un error que encontré fue durante el desarrollo de Insertion Sort. En una de las versiones tenía una condición donde se hacía una comparación con una posición del arreglo antes de comprobar que el índice siguiera siendo válido, esto podía hacer que el programa intentara acceder a una posición que no existía cuando el índice llegaba al límite. Esta fue una sugerencia que inicialmente me dio Copilot, pero al revisarla me di cuenta de que estaba mal. Para corregirlo cambié el orden de la condición para que primero se comprobara que el índice fuera válido y después se hiciera la comparación.

También utilicé ChatGPT para organizar un poco mis ideas antes de empezar algunas partes. Me gusta primero hacer como un draft de “ok, primero tengo que hacer esto y después esto”. Hago esto en una libreta organizando mis ideas y preguntando a Chat si voy por un buen camino y a partir de ahí empiezo a trabajar. Por ejemplo, también me ayudó con el README porque al inicio no sabía muy bien qué debía incluir, entonces pregunté cómo era un README básico y a partir de eso hice el mío.

Con todo esto pude darme cuenta de que la inteligencia artificial me ayuda mucho al momento de programar, pero obviamente no siempre lo que propone está completamente bien. Aunque pueda generar mucho código en 5 segundos, puede tener errores o no entender completamente lo que quiero hacer, por eso traté de darme el tiempo de entender bien las instrucciones, revisar la lógica del código y entender lo que estaba haciendo, en lugar de simplemente darle toda la actividad a ChatGPT y pedirle que la hiciera.